

Article

ToR-Lite: A Lightweight Semantic Query Decomposition for Multi-Hop Retrieval-Augmented Generation in Cloud-Based AI Systems

Hee-Kyong Yoo , Wonbae Kim  and Nammee Moon *

Department of Convergence Engineering, Hoseo University, Seoul 06724, Republic of Korea; hkyoo@dataslab.co.kr (H.-K.Y.); wonkim@dataslab.co.kr (W.K.)

* Correspondence: mnm@hoseo.edu

Featured Application

ToR-Lite is directly applicable to latency-sensitive, cost-constrained cloud AI services—including enterprise knowledge bases, legal and clinical document retrieval, and public-sector AI initiatives such as government document search and intelligent civil service portals—where LLM-based query decomposition is operationally prohibitive.

Abstract

Cloud-based AI systems increasingly rely on Retrieval-Augmented Generation (RAG) to handle complex, knowledge-intensive queries. However, query decomposition for multi-hop retrieval—traditionally powered by large language models (LLMs)—incurs significant latency and cost, rendering it impractical for large-scale, cost-sensitive cloud deployments. We propose ToR-Lite, a lightweight, generative LLM-free semantic query decomposition framework designed to enhance multi-hop retrieval efficiency in cloud-based AI systems. ToR-Lite employs a novel Word-Window Splitting algorithm that detects semantic break-points via sliding window embeddings, effectively decomposing complex queries without expensive LLM inference. Experiments on the MultiHop-RAG benchmark ($n = 2255$) demonstrate that ToR-Lite achieves +6.03 pp Hits@10 and +0.89 pp Exact Match improvements over the Baseline, while operating 3.18 times faster than LLM-based Adaptive ToR. Retrieval performance correlates monotonically with decomposition granularity: three sub-query decompositions ($\#Dq = 3$) yields a +7.00 pp Hits@10 improvement, confirming that semantic granularity is a key driver of retrieval performance. Comparison with rule-based Baselines confirms that these gains derive from the precision of semantic boundary detection rather than decomposition quantity alone. ToR-Lite delivers nearly twice the retrieval improvement per unit of computational cost, offering a practical and cost-effective solution for latency-sensitive cloud AI deployments.



Academic Editor: Jisu Park

Received: 2 March 2026

Revised: 13 April 2026

Accepted: 17 April 2026

Published: 19 April 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Keywords: lightweight query decomposition; LLM-free retrieval-augmented generation; multi-hop information retrieval; semantic segmentation; real-time query processing; cloud AI systems; cost-efficient AI services

1. Introduction

1.1. Background and Motivation

The rapid evolution of cloud computing has positioned it as the backbone of modern artificial intelligence (AI) services, enabling the scalable deployment of large language models

(LLMs) across diverse industries. As AI systems are integrated into critical infrastructure—from intelligent vehicular networks to smart city management—the convergence of AI and cloud technologies has become pivotal for empowering future technological developments [1]. This trend is further supported by emerging approaches such as energy-efficient hierarchical federated learning enabling scalable AI deployment across distributed IIoT environments [2]. Cloud environments provide a cost-effective way for organizations to allocate computing resources dynamically over the Internet for virtualized infrastructure such as servers, storage, and networks to meet each organization's unique operational requirements [1]. Recent research on AI systems has demonstrated the importance of decision support mechanisms and efficient retrieval architectures for scalable cloud deployments, with the majority of enterprises adopting hybrid cloud models for their AI workloads [3,4].

Within this landscape, Retrieval-Augmented Generation (RAG) has emerged as a dominant paradigm to address the inherent limitations of LLMs, such as outdated knowledge and the tendency to hallucinate. By retrieving relevant external documents before generating a response, RAG systems aim to ground LLM outputs in factual evidence, thereby enhancing reliability and trustworthiness [5]. This approach has proven particularly valuable in knowledge-intensive applications where factual accuracy is paramount, including clinical summarization, legal document analysis, and financial reporting [3,6].

However, the development of RAG systems in cloud environments introduces significant operational challenges: characteristics inherent in cloud computing—such as remote data processing, shared tenant environments, and dynamic scalability—give rise to complex considerations including latency optimization, API cost management, and computational resource allocation [4,7]. Furthermore, the increasing adoption of LLM-powered applications has amplified concerns about operational costs, as each API call to large models incurs measurable expenses that accumulate rapidly in production environments handling thousands of queries daily [1,8].

1.2. Challenges in Multi-Hop Query Processing

Despite the advancements in RAG technology, standard RAG systems often struggle with “multi-hop” queries—complex questions that require aggregating information from multiple disjoint sources to derive an answer. Traditional single-step retrieval methods frequently fail to capture the full semantic scope of such queries, leading to incomplete context retrieval and subsequent reasoning errors [9,10]. Consider, for example, a query such as “Analyze the impact of 2023 tech layoffs on the AI industry and examine the ethical concerns and regulatory challenges surrounding artificial intelligence development.” This question encompasses at least two distinct semantic domains: economic and business implications on one hand, and ethical and legal considerations on the other. When processed as a single unit, the resulting embedding vector represents a diluted average of these disparate semantic components, often failing to retrieve evidence that precisely addresses either sub-topic [11,12]. The embedding dilution problem constitutes the primary retrieval failure mode for multi-hop queries.

To address this limitation, recent research has focused on query decomposition techniques, which break down complex queries into simpler, sequential sub-queries. Approaches such as Least-to-Most Prompting [13], Chain-of-Thought reasoning [14], and Tree-of-Retrieval (ToR) have demonstrated significant improvements in handling multi-hop questions by enabling more targeted evidence retrieval for each semantic component [15,16]. Our previous work on ToR-RAG [17], illustrated in Figure 1, established that hierarchical decomposition of queries into semantic sub-units substantially improves retrieval coverage, as illustrated in Figure 2. Adaptive ToR [18] extended this architecture through a Query Tree Classifier that routes queries along dual paths—simple queries to single-step retrieval

and complex queries to hierarchical tree expansion—with an Adaptive Pruning Module filtering low-relevance candidates via quantitative similarity thresholds combined with LLM-based quality evaluation, requiring between 2 and 20 LLM calls per query depending on complexity.

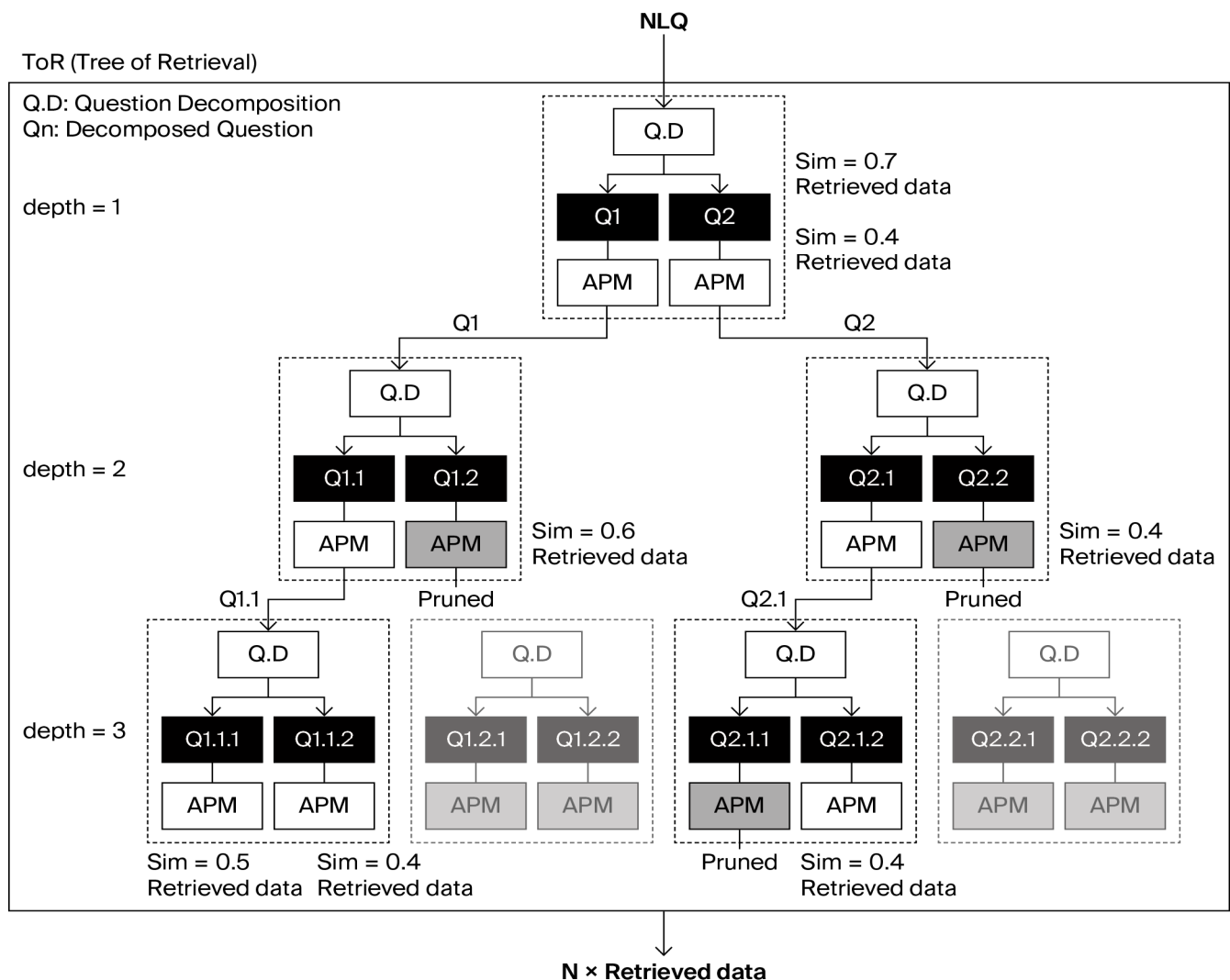


Figure 1. Tree-of-Retrieval (ToR) architecture (based on [17]). Each node represents a WH-form binary sub-query evaluated by an LLM-as-a-Judge module across four criteria (sufficiency, non-redundancy, consistency, decomposition quality), with recursive expansion up to Depth = 3.

The foregoing analysis reveals a clear research gap: all existing query decomposition methods universally depend on LLM inference during the decomposition phase (Table 1), imposing latency accumulation, API cost overhead, and recursive hallucination risk in cloud deployments. No existing method has demonstrated that effective query decomposition can be achieved through deterministic, LLM-free analysis of embedding representations. ToR-Lite is designed to close this gap. ToR-Lite replaces LLM-based decomposition entirely with embedding space geometric segmentation, reserving a single LLM call exclusively for answer generation.

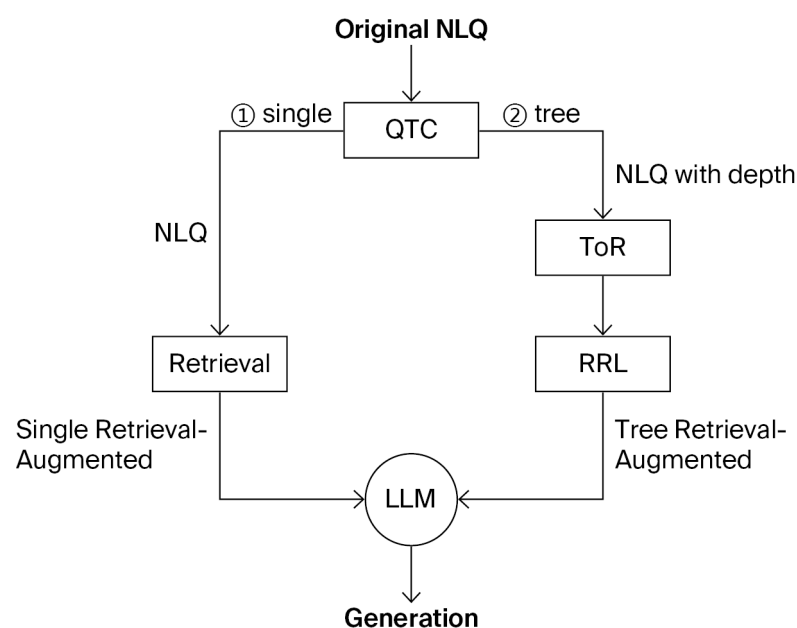


Figure 2. Overall architecture of the Adaptive ToR system (based on [18]). A Query Tree Classifier routes simple queries to single-step retrieval and complex queries to hierarchical tree expansion. The Adaptive Pruning Module filters low-relevance candidates via similarity thresholds and LLM-based evaluation, requiring 2–20 LLM calls per query.

Table 1. Systematic comparison of query decomposition methods.

Method	LLM Required	Decomposition Type	Latency Profile	Multi-Hop Support	Deterministic	Training Required
Least-to-Most [13]	Yes (per Step)	Sequential Sub-problem	High (iterative LLM)	Yes	No	No
Chain-of-Thought [14]	Yes (single)	Implicit Reasoning chain	Medium	Partial	No	No
IRCoT [15]	Yes (iterative)	Interleaved Retrieval + COT	High (multi-round)	Yes	No	No
Self-Ask [16]	Yes (iterative)	Follow-up questions	High (recursive LLM)	Yes	No	No
POQD [19]	Yes (trained model)	Policy-based decomposition	Medium (inference)	Yes	No	Yes
Query2 DoC [20]	Yes (single)	Query expansion	Medium	Partial	No	No
ToR-RAG [17]	Yes (per-node)	Hierarchical Tree decomposition	High (fixed-depth)	Yes	No	No
Adaptive ToR [18]	Yes (selective)	Complexity-Aware Dynamic tree decomposition	High (adaptive-depth)	Yes	No	No
ToR-Lite	No	Embedding Space Geometric Segmentation	Low (deterministic Single LLM)	Yes	Yes	No

While these decomposition methods have proven effective, they introduce a significant bottleneck in cloud-based deployments: the repeated invocation of large language models for intermediate processing steps dramatically increases latency and operational costs [7,9].

Furthermore, LLM-based decomposition introduces a recursive risk of hallucination; if the model errs in breaking down the query into WH-form binary subqueries, the subsequent retrieval and reasoning steps are fundamentally compromised [5,9]. This “cascading hallucination” phenomenon can derail the entire retrieval process when the decomposition

model generates sub-questions about non-existent entities or misinterprets the user's intent [3,21].

1.3. Research Objectives and Approach

In this paper, we introduce ToR-Lite, a lightweight semantic query decomposition framework designed specifically for efficient multi-hop RAG in cloud environments. ToR-Lite represents the third iteration in our research progression, following the structural foundation of ToR-RAG [17] and the dynamic depth adaptation of Adaptive ToR [18]. Unlike its predecessors, ToR-Lite completely eliminates the use of LLMs during the decomposition phase. Instead, it utilizes a novel Word-Window Splitting algorithm based on embedding similarity to identify semantic boundaries within a query. This approach leverages the observation that complex multi-hop queries typically exhibit measurable semantic discontinuities at topic transition points—discontinuities that can be detected through geometric analysis of the query's embedding space without requiring generative inference [12,22].

The core innovation of ToR-Lite lies in its adaptation of document segmentation techniques to the micro-scale of user queries. Traditional text segmentation approaches, dating back to the TextTiling algorithm [23], have long used lexical cohesion scores to identify topic boundaries in long documents. ToR-Lite modernizes this concept by applying dense vector embeddings to short query texts, using sliding window similarity analysis to detect “valleys” of semantic similarity that indicate topic shifts [24]. By identifying these natural breakpoints, the system decomposes complex queries into coherent sub-queries that better align with specific evidence passages in the vector database, thereby improving retrieval precision without the computational overhead of LLM-based decomposition [11,25]. The key technical adaptations—word-level granularity and the adaptive threshold formulation—are detailed in Sections 1.4 and 2.4.

This lightweight approach aligns with the growing emphasis on resource-efficient AI systems, capturing the benefits of query decomposition—improved retrieval coverage and reduced hallucination risk—while maintaining computational costs comparable to standard single-step RAG [8,19].

1.4. Contributions

This study makes four key contributions:

- (i) We propose ToR-Lite, an LLM-free framework that leverages lightweight embedding models to decompose multi-hop queries, addressing the cost-latency trade-off inherent in cloud AI systems [4,7,26]. To the best of the authors' knowledge, ToR-Lite is the first query decomposition method that operates entirely without LLM inference during the decomposition phase.
- (ii) We introduce the Word-Window Splitting algorithm, a semantic segmentation technique that dynamically identifies topic transitions within a query using sliding window embeddings and an adaptive statistical threshold ($\tau = \mu - 2\sigma$), generating coherent sub-queries without generative inference [12,23,24]. The algorithm extends the TextTiling paradigm [23] to the micro-scale of short queries by replacing bag-of-words representations with dense vector embeddings, and generalizes the threshold formulation from sentence-level to word-level granularity.
- (iii) We demonstrate through extensive experimentation on the MultiHop-RAG benchmark [27] ($n = 2255$) that ToR-Lite achieves an Exact Match improvement of +0.89 percentage points and a Hits@10 improvement of +6.03 percentage points over the Baseline. Retrieval performance exhibits a monotonic correlation with decomposition granularity. Comparison with three rule-based decomposition Baselines confirms that

these gains are attributable to the semantic precision of boundary detection rather than to the act of decomposition in itself [9,10,21].

- (iv) We validate the computational efficiency of ToR-Lite, demonstrating that it operates 3.18 times faster than LLM-based adaptive methods while maintaining a single LLM call for final answer generation, delivering approximately 1.9 times the retrieval improvement per unit of computational cost, and offering a cost-effective solution for real-time cloud AI applications [8,18,19].

The methodological contributions of this paper are strictly limited to two components:

(a) the Word-Window Splitting algorithm and **(b) the LLM-free decomposition framework architecture**. All other system components—including embedding model selection, reranking strategy, and pruning threshold calibration—are experimental design decisions made to construct a rigorous and reproducible evaluation environment. This distinction ensures that the performance gains reported in Section 4 are attributable solely to the decomposition mechanism rather than to ancillary engineering choices.

The remainder of this paper is organized as follows. Section 2 reviews related work in RAG and query decomposition. Section 3 details the proposed ToR-Lite architecture and the Word-Window Splitting algorithm. Section 4 presents the experimental setup and results. Section 5 discusses the implications of our findings in the context of cloud AI deployment. Finally, Section 6 concludes the paper with future research directions.

2. Related Work

2.1. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation has emerged as a foundational paradigm for knowledge-intensive natural language processing tasks, combining the parametric memory encoded in pre-trained model weights with non-parametric memory accessed through external knowledge indices. The seminal work by Lewis et al. [5] demonstrated that augmenting sequence-to-sequence models with a dense retrieval component significantly improves performance on open-domain question answering, fact verification, and knowledge-grounded dialogue tasks. This approach addresses two critical limitations of standalone LLMs: the inability to update knowledge without retraining and the tendency to generate plausible but factually incorrect responses [9,21].

The technical foundation of RAG systems rests on dense passage retrieval mechanisms that encode both queries and documents into shared vector spaces, enabling efficient similarity-based retrieval [11]. The introduction of efficient approximate nearest neighbor search algorithms, such as Hierarchical Navigable Small World (HNSW) graphs, has further enabled the deployment of dense retrieval at scale in production environments [25]. Recent comprehensive surveys [6,9] have categorized the evolution of RAG into three paradigms: naïve RAG, advanced RAG, and modular RAG. Naïve RAG follows a simple retrieve-then-generate pipeline, while advanced RAG incorporates pre-retrieval and post-retrieval optimization strategies, including query rewriting, document reranking, and context compression [20,28]. Modular RAG represents the most recent evolution, decomposing the RAG pipeline into interchangeable components that can be flexibly combined for specific use cases [8,21].

Within this taxonomy, ToR-Lite contributes primarily to the pre-retrieval optimization category by restructuring the query before it reaches the vector database. By decomposing complex queries into semantically coherent sub-units, ToR-Lite ensures that each retrieval operation targets a specific information need, thereby improving the precision and relevance of retrieved passages without requiring additional LLM inference during the decomposition phase [17,19].

ToR-Lite operates at the pipeline level, eliminating LLM inference during query decomposition. A complementary line of research targets model-level efficiency through techniques such as quantization, distillation, and pruning, reducing the cost of individual LLM calls. These two dimensions are orthogonal and can be combined synergistically, e.g., deploying a quantized LLM for ToR-Lite's single generation call.

2.2. Multi-Hop Query Processing

Multi-hop QA represents a particularly challenging subset of knowledge-intensive tasks, requiring systems to aggregate and reason over information distributed across multiple documents [10]. Unlike single-hop questions that can typically be answered from a single evidence source, multi-hop queries demand the identification and synthesis of multiple pieces of evidence, often connected through implicit relationships or shared entities [15,21]. This complexity poses significant challenges for standard dense retrieval methods, as the semantic embedding of a complex question may not align well with the embeddings of individual evidence paragraphs [12,29].

Several factors contribute to the difficulty of multi-hop retrieval. The embedding dilution problem arises when a complex query containing multiple semantic components is encoded into a single vector representation, potentially failing to match strongly with evidence passages that address specific sub-components [22,29]. The evidence distribution challenge occurs when relevant information is scattered across documents without obvious lexical or semantic connections [21,30]. The MultiHop-RAG benchmark [27], recently introduced to evaluate RAG systems on multi-hop reasoning tasks, has highlighted substantial performance gaps between single-hop and multi-hop retrieval, underscoring the need for specialized approaches.

To address these challenges, researchers have explored query decomposition approaches that explicitly break down multi-hop questions into sequences of simpler questions [13,16], iterative retrieval methods that alternate between retrieval and reasoning steps [14,15], and graph-based approaches that construct explicit reasoning graphs capturing relationships between entities and evidence passages [30,31]. ToR-Lite addresses multi-hop retrieval through embedding space analysis rather than LLM-guided decomposition, leveraging geometric properties to identify natural semantic boundaries within complex queries, splitting them into granular sub-queries that produce more targeted embedding vectors for specific evidence passages [17,19,24].

2.3. LLM-Based Query Decomposition

The application of large language models to query decomposition has yielded significant advances in multi-hop question answering. Chain-of-Thought prompting [14] encourages LLMs to generate intermediate reasoning steps, implicitly decomposing complex problems into sequential cognitive operations. Least-to-Most Prompting [13] extends this by explicitly instructing models to decompose problems into sub-problems of increasing complexity. Self-Ask [16] prompts LLMs to generate follow-up questions that provide information necessary to address the original query, while IRCoT [15] interleaves retrieval operations with Chain-of-Thought reasoning. Query rewriting approaches [20,28] use LLMs to reformulate questions for improved retrieval performance through multiple query variations or expansions.

Our previous work established the Tree-of-Retrieval (ToR) framework for multi-hop RAG. ToR-RAG [17] introduced hierarchical query decomposition through tree-structured retrieval, demonstrating effectiveness in complex question processing by expanding queries into a binary sub-query with LLM-based quality evaluation at each node. Adaptive ToR [18] subsequently introduced complexity-aware dynamic depth control, employing a Query

Tree Classifier to route queries through dual paths—simple queries to single-step retrieval and complex queries to hierarchical exploration—while an Adaptive Pruning Module filters low-relevance candidates using quantitative similarity thresholds combined with LLM-as-a-Judge evaluation [8].

However, LLM-based decomposition methods share a fundamental limitation: the computational overhead imposed by repeated model invocations [7,26]. Each LLM call introduces latency, monetary cost proportional to token consumption, and potential for error propagation [4,9]. In Adaptive ToR, the LLM-based controller requires between 2 and 20 calls per query with average latency approaching ≈ 8.1 s—a computational profile limiting applicability in cost-sensitive production environments [1,3]. Furthermore, LLM-based decomposition introduces recursive hallucination risk where erroneous sub-questions can derail subsequent retrieval and reasoning steps [5,21]. ToR-Lite addresses these limitations by replacing LLM-based decomposition with a purely geometric approach operating directly on embedding representations [12,19,22].

To position ToR-Lite explicitly within this landscape, Table 1 presents a systematic methodological comparison across nine methods, including external approaches from the literature and all three works in the authors’ research lineage.

As Table 1 demonstrates, all existing query decomposition methods—including those from the broader literature and the authors’ own prior work—universally rely on LLM inference during the decomposition phase. Notably, the authors’ three prior studies were evaluated on different benchmarks, reflecting distinct research objectives—ToR-RAG [17] and ToR-Lite target multi-hop retrieval decomposition—and are evaluated on MultiHop-RAG [27], while Adaptive ToR [18] targets complexity-aware intent classification in task-oriented dialogue and is evaluated on NLU++ [32]. **These benchmarks address structurally orthogonal problems and are not interchangeable.** ToR-Lite is the only method in this comparison that eliminates LLM inference entirely from the decomposition phase, operating through purely geometric analysis of embedding representations.

Regarding a direct quantitative comparison with LLM-based external methods (POQD, IRCOT, Self-Ask, etc.), these methods have not been evaluated on the MultiHop-RAG benchmark in the literature, and controlled reproduction would introduce confounding variables—including differing generator models, retrieval configurations, and embedding choices—that would compromise experimental validity. The comparison in Table 1 is therefore structured as a **methodological characteristic comparison** rather than a quantitative performance comparison. Quantitative comparison on MultiHop-RAG is available for Adaptive ToR, re-implemented by the authors under identical conditions following the architecture in Table 2.

Table 2. Four embedding-reranker configurations for robustness evaluation.

Configuration	Embedding	Reranker
BGE + Cohere	BGE-base-en-v1.5 (768d; BAAI, Beijing, China)	Cohere rerank-english-v3.0 (Cohere Inc., Toronto, ON, Canada)
BGE + msmarco	BGE-base-en-v1.5	cross-encoder/ms-marco-MiniLM-L-6-v2 (UKP Lab, TU Darmstadt, Germany)
Cohere + Cohere	Cohere embed-english-v3.0 (1024d; Cohere Inc., Toronto, ON, Canada)	Cohere rerank-english-v3.0
Mpnet + msmarco	all-mpnet-base-v2 (768d; UKP Lab, Germany)	cross-encoder/ms-marco-MiniLM-L-6-v2

2.4. Embedding-Based Semantic Segmentation

The concept of segmenting text based on semantic coherence has a rich history in computational linguistics. The TextTiling algorithm [23] pioneered automatic topic seg-

mentation by analyzing lexical cohesion patterns across adjacent text blocks, identifying topic boundaries at points where similarity drops significantly. More recent approaches have leveraged dense vector representations to capture semantic relationships transcending lexical overlap [11,22]. Sentence embedding models such as Sentence-BERT [12] enable computation of semantic similarity between arbitrary text spans, providing a foundation for coherence analysis superior to traditional bag-of-words representations.

The application of semantic segmentation to query processing represents a relatively unexplored direction. While substantial research has addressed segmentation of long documents and dialogues [9,24], short multi-intent queries pose distinct challenges: limited context for semantic analysis within 10–50 tokens, subtle linguistic cues signaling topic transitions, and requirements for grammatically well-formed sub-units suitable for independent retrieval [20,28]. POQD [19] represents a recent approach employing a policy model trained to decompose queries for optimal retrieval performance, though this requires task-specific training data and introduces computational overhead.

ToR-Lite modernizes the TextTiling concept for application to short query texts using dense vector embeddings. The Word-Window Splitting algorithm generates sliding window embeddings centered at each token position, computing cosine similarity between adjacent windows to measure semantic continuity [12,29]. Points where similarity drops below a dynamically computed threshold are identified as potential topic boundaries. The key adaptations introduced by ToR-Lite are as follows: (i) the granularity of the window unit is reduced from the sentence level to the word level, enabling the detection of topic transitions within short queries of 10–50 tokens where sentence-level analysis would be too coarse; and (ii) the threshold formulation is generalized from a fixed percentile to the adaptive $\tau = \mu - s\sigma$ formulation.

These connections position the Word-Window Splitting algorithm as a principled adaptation of established segmentation theory to the micro-scale of user queries, rather than an ad hoc engineering decision.

2.5. Positioning of ToR-Lite

The research lineage from ToR-RAG through Adaptive ToR to ToR-Lite represents progressive refinement along the performance–efficiency frontier of tree-based retrieval. ToR-RAG [17] established the effectiveness of hierarchical query decomposition but employed fixed-depth expansion regardless of query complexity. Adaptive ToR [18] introduced dynamic depth control through LLM-based complexity assessment, achieving superior accuracy at sustained LLM dependency—an average latency approaching 8099 ms per query with 2–20 LLM calls. ToR-Lite completes this progression by demonstrating that effective decomposition can be achieved through lightweight geometric methods while preserving the core benefits of structured retrieval [11,22,29]. Quantitatively, ToR-Lite captures approximately 44% of Adaptive ToR’s Hits@10 gain while operating at one-third the latency, delivering nearly twice the retrieval improvement per unit of computational cost, as detailed in Sections 4.1 and 4.4.

3. Materials and Methods

3.1. Overall System Architecture

The ToR-Lite framework is designed to optimize multi-hop retrieval efficiency while eliminating the computational overhead of LLM-based query decomposition. As illustrated in Figure 3, the system integrates four primary modules: the Query Decomposer Module (QDM), the Adaptive Pruner Module (APM), the Relevance-based Retrieval Layer (RRL), and the LLM generator.

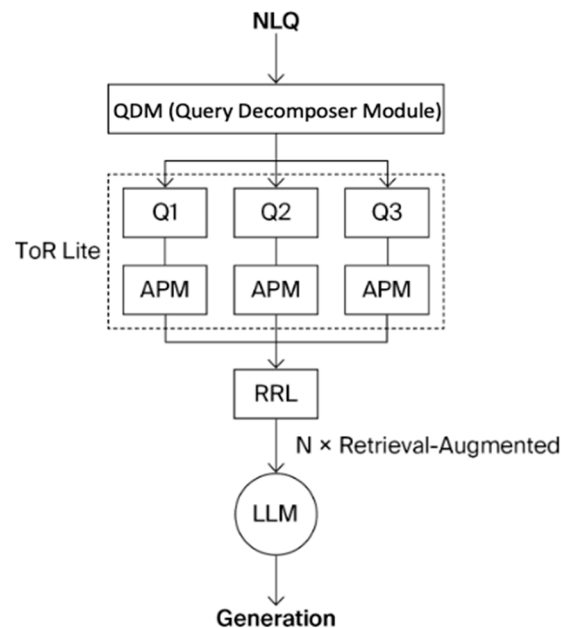


Figure 3. ToR-Lite architecture overview.

The QDM receives the original user query and applies the Word-Window Splitting algorithm (Section 3.2) to produce N sub-queries ($1 \leq N \leq 3$). Each sub-query is independently processed in parallel by the Adaptive Pruner Module (APM), which filters low-relevance passages based on cosine similarity thresholds (Section 3.4). The surviving chunks from each sub-query are then forwarded to the Relevance-based Retrieval Layer (RRL), which aggregates, deduplicates, and reranks candidates by similarity score. The reranked evidence is finally passed to the LLM generator alongside the original query to produce the final answer (Section 3.5). The pipeline flow QDM $\rightarrow$ APM $\rightarrow$ RRL $\rightarrow$ Generator—representing sub-query generation, parallel chunk filtering, similarity-based reranking, and answer generation, respectively—is explicitly represented in Figure 3 with directional arrows and data-flow annotations.

Unlike standard RAG, which retrieves once with potentially diluted query embedding, ToR-Lite executes parallel retrieval for decomposed semantic units. This design preserves the retrieval benefits of query decomposition—improved evidence coverage for multi-hop questions—while maintaining a computational efficiency below that of LLM-based decomposition methods.

3.2. Word-Window Splitting Algorithm

The core innovation of ToR-Lite is the Word-Window Splitting algorithm, which detects semantic boundaries using pre-trained embedding models without requiring LLM inference. The algorithm identifies topic transitions through geometric analysis of the query’s embedding space. The process involves the following five steps.

(1) Tokenization and Windowing.

The input query Q is tokenized into a sequence of words. A sliding window S_t centered at each word position t is defined with window size $W:W = 7$ for short queries (<40 words) to capture fine-grained semantic shifts, and $W = 9$ for longer queries (≥ 40 words) to smooth out noise and prevent over-segmentation. Figure 4 illustrates this sliding window mechanism.

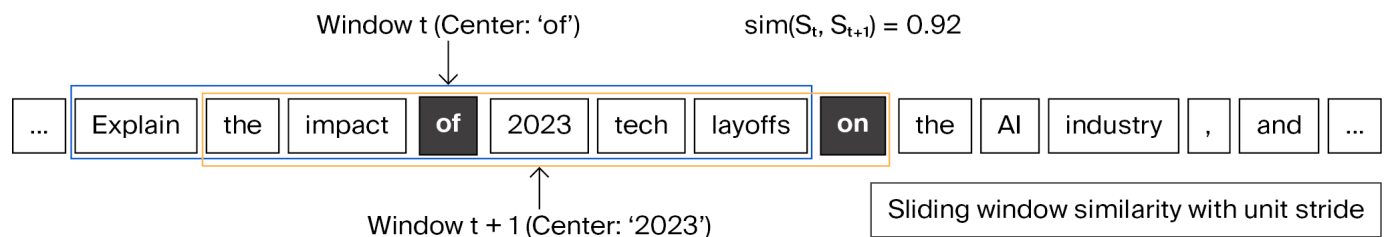


Figure 4. Sliding Word-Window mechanism.

(2) Embedding Generation.

For each window position, a dense vector embedding $E(S_t)$ is generated using a lightweight encoder, BGE-base-en-v1.5 (BAAI, Beijing Academy of AI, Beijing, China), capturing the local semantic context surrounding each word position and enabling detection of topic shifts within the query.

(3) Similarity Calculation.

For each position, t ($1 \leq t \leq L - 1$, where L is the number of words in the query), the cosine similarity between adjacent window embeddings measures semantic continuity:

$$\text{Sim}(S_t, S_{t+1}) = \frac{E(S_t) \cdot E(S_{t+1})}{\|E(S_t)\| \times \|E(S_{t+1})\|} \quad (1)$$

Here, S_t denotes the sliding window of size W centered at word position t (as defined in Step 1); $E(S_t) \in \mathbb{R}^{768}$ is the dense vector produced by BGE-base-en-v1.5 (as defined in Step 2), $(\cdot)$ denotes the inner product, and $\|\cdot\|$ the L_2 norm. High similarity scores (typically 0.7–0.9) indicate topical coherence between adjacent windows, while sharp drops signal potential topic transitions.

(4) Valley Detection.

Similarity valleys are identified as potential split points using a dynamic threshold τ :

$$\tau = \mu - s\sigma \quad (2)$$

where μ is the mean similarity across all adjacent window pairs in the query, σ is the standard deviation, and s is a sensitivity parameter (empirically set to $s = 2.0$). A split occurs at position t if $\text{Sim}(t, t + 1) < \tau$. The adaptive $\tau = \mu - s\sigma$ formulation is adopted in preference to a fixed threshold because a fixed value (e.g., $\tau = 0.8$) cannot adapt to query-level variation in similarity scale: short queries tend to produce globally lower similarity scores, while longer queries produce a higher Baseline similarity.

The $\tau = \mu - s\sigma$ formulation normalizes against each query's own similarity distribution, directly analogous to LlamaIndex's breakpoint_percentile_threshold (LlamaIndex Inc., San Francisco, CA, USA), where percentile = 95 corresponds approximately to $\mu - 2\sigma$ (i.e., the lower ~2.3% of a normal distribution), and $s = 2.0$ is confirmed as empirically optimal through the ablation study in Section 4.5. Figure 5 visualizes this valley detection process.

(5) Boundary Alignment.

Detected split points are snapped to the nearest natural linguistic boundaries—such as commas, semicolons, or conjunctions (“and”, “but”, “or”)—informed by syntactic dependency structures [33,34] and implemented using industrial-strength NLP toolkits [35], to ensure the grammatical correctness of the resulting sub-queries. A No Trail Guard mechanism additionally prevents sub-queries from ending with function words such as articles or prepositions, performing cascading corrections when necessary. Figure 6 illustrates the complete query decomposition procedure.

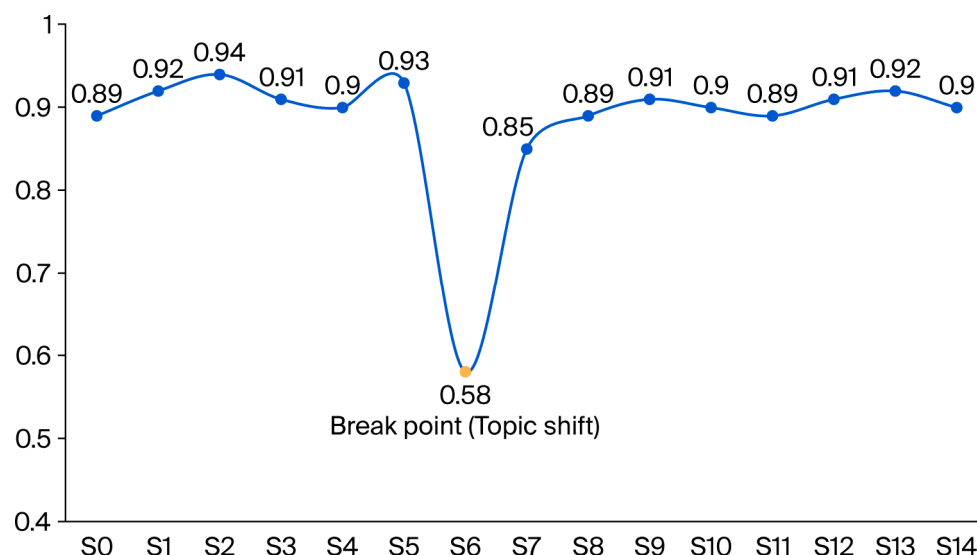


Figure 5. Adjacent window similarity graph with breakpoint detection.

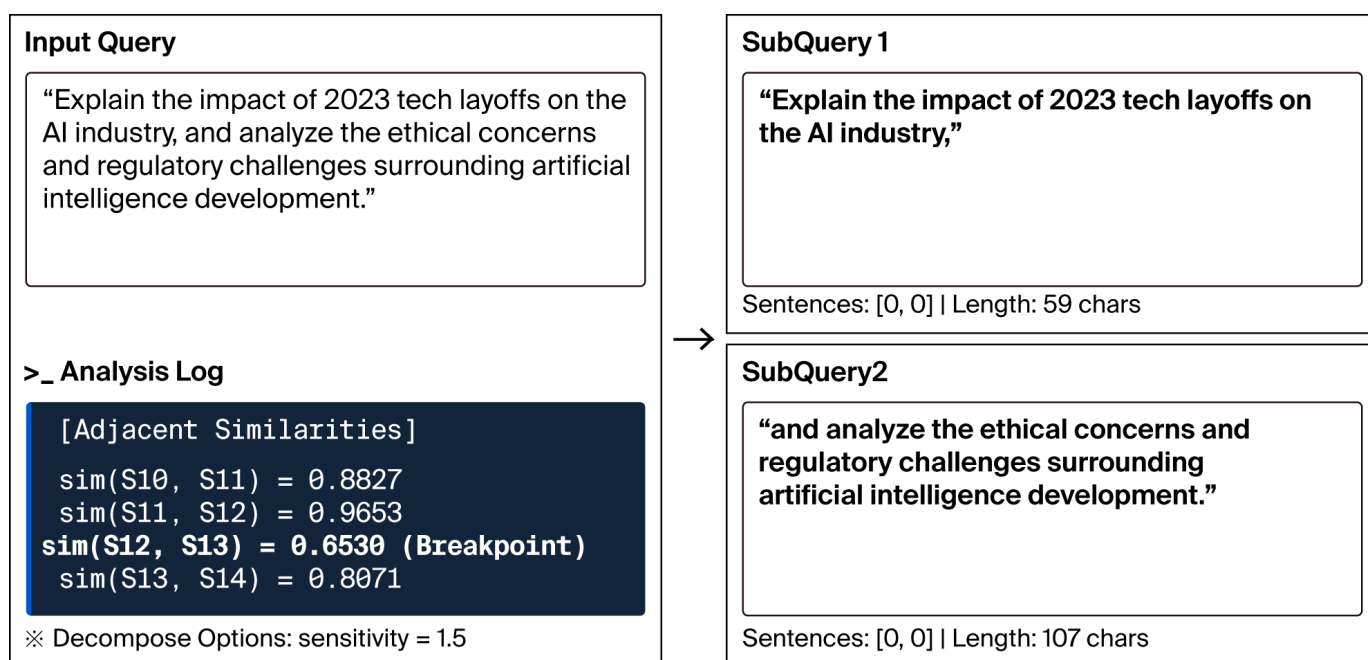


Figure 6. Query decomposition procedure. Note: The first “and” coincides with a similarity valley ($\text{sim}(S12, S13) = 0.6530$, Breakpoint) and is selected as the split boundary; the second “and” ($\text{sim}(S13, S14) = 0.8071$, above τ) is preserved within SubQuery 2.

The algorithm naturally adapts to query complexity without explicit classification: simple queries with uniform semantic content produce no breakpoints and result in a single sub-query ($N = 1$), while complex multi-intent queries exhibit detectable similarity valleys that trigger decomposition into two or three sub-queries ($N = 2$ or 3).

Clarification on conjunction handling. Conjunctions such as “and”, “but”, and “or” do not function as automatic split triggers. The algorithm determines split positions exclusively through the valley detection mechanism (step 4); conjunctions serve only as alignment anchors in the subsequent boundary alignment step—selected as a split boundary only when nearest to a position where $\text{Sim}(t, t + 1) < \tau$. Figure 6 illustrates this distinction. The input query contains two instances of “and”. At the first instances—connecting two independent clauses on distinct topics— $\text{sim}(S12, S13) = 0.6530$ falls below τ ,

triggering a Breakpoint that is snapped to this conjunction during boundary alignment. At the second instance—connecting parallel noun phrases (“ethical concerns” and “regulatory challenges”) within the same semantic domain— $\text{sim}(S_{13}, S_{14}) = 0.8071$ remains above τ , producing no valley and no split. The identical conjunction thus receives opposite treatment based solely on the local similarity profile, confirming that split decisions are driven embedding-space semantics rather than surface-level syntactic cues.

3.3. Adaptive Window Configuration

The window size W controls the granularity of local semantic context capture. For short queries (<40 words), $W = 7$ detects fine-grained topic transitions within narrow spans; for longer queries (≥ 40 words), $W = 9$ smooths out surface-level phrasing variations that could trigger false breakpoints.

Design rationale. The window size W is determined by the buffer parameter as $W = 2 \cdot \text{buffer} + 1$, with $\text{buffer_size} = 3$ ($W = 7$) for queries below 40 words and $\text{buffer_size_long} = 4$ ($W = 9$) for longer queries. At edge positions, where insufficient neighboring words exist, the window degrades gracefully to 4–6 words rather than padding with artificial tokens. The choice of $W = 7$ follows the convention established in production semantic chunking frameworks such as LlamaIndex’s SemanticSplitterNodeParser, which similarly employs short context windows centered on each segmentation candidate to balance local sensitivity with statistical reliability of cosine similarity estimates. Smaller buffer values (e.g., $\text{buffer} = 1$, yielding $W \leq 3$) would render similarity computations dominated by individual lexical matches rather than contextual semantics, while larger buffers (e.g., $\text{buffer} \geq 5$, yielding $W \geq 11$) would smooth across multiple semantic units and obscure the topic boundaries the algorithm aims to detect. The 40-word threshold is a default value that empirically distinguishes short queries (where finer-grained windows benefit from local sensitivity) from longer ones (where wider windows mitigate phrasing variability). The threshold is configurable for adaptation to corpora with different query length distributions.

Robustness. The algorithm’s primary tunable parameter is the threshold sensitivity s in $\tau = \mu - s\sigma$, rather than the window size itself. The s ablation reported in Section 4.5 demonstrates that performance varies by less than 1 percentage point in Hits@10 across $s \in \{1.0, 1.5, 2.0, 2.5, 3.0\}$, indicating that the adaptive threshold normalization absorbs much of the variation that fixed-window alternatives would expose. A formal grid search over W is left to future work; the present results establish the baseline performance of the $W = 7/9$ configuration on which all reported metrics are based.

3.4. Adaptive Pruner Module

The architecture of the APM and its interface with the RRL is illustrated in Figure 7.

Decomposing a query into multiple sub-queries increases the volume of retrieved documents, potentially introducing noise that can mislead the generator and increase hallucination risks. The Adaptive Pruner Module (APM) addresses this challenge by acting as a quality-based gatekeeper that filters low-relevance passages before they reach the generation stage.

For each retrieved chunk C_i , the APM calculates its relevance score R_i against its corresponding sub-query using cosine similarity in the embedding space. If R_i falls below a predefined threshold θ , the chunk is discarded from the evidence pool.

$$\text{Keep}(C_i) = \begin{cases} \text{True} & \text{if } R_i \geq \theta \\ \text{False} & \text{if } R_i < \theta \end{cases} \quad (3)$$

The threshold θ is calibrated to achieve a 60–70% chunk retention rate within each embedding model’s similarity score distribution: $\theta = 0.68$ for BGE embeddings, $\theta = 0.55$ for Cohere embeddings, and $\theta = 0.52$ for mpnet embeddings. These values represent experimental design decisions rather than methodological contributions.

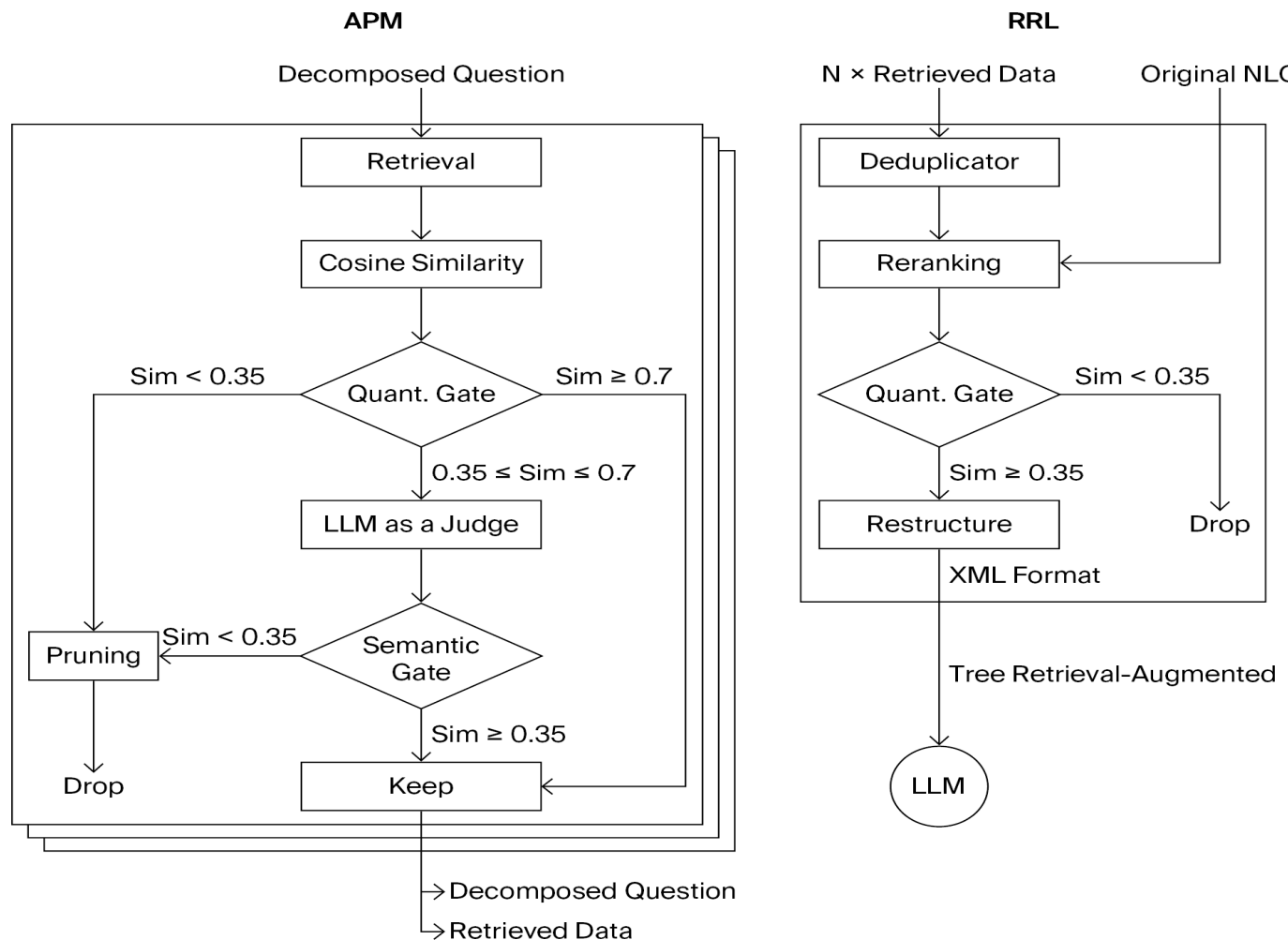


Figure 7. The architecture of APM and RRL.

The APM operates at the individual chunk level: each sub-query may contribute between zero and k chunks to the downstream evidence pool. Chunks from all sub-queries compete equally in the subsequent unified reranking stage (Section 3.5), ensuring that globally relevant evidence from any sub-query can reach the final generation prompt regardless of how many chunks that sub-query contributed.

For the edge case where all chunks from all sub-queries are pruned (observed in 122 queries, 5.4% of the dataset), the system falls back gracefully to the initial retrieval results prior to decomposition, operating equivalently to the Baseline. This graceful degradation mechanism ensures that decomposition failure does not produce a worse outcome than no decomposition; $\Delta\text{Hits@10} \approx 0$ for this fallback subgroup confirms safe behavior.

3.5. Context Integration and Reranking

Following the pruning stage, the remaining chunks from all sub-queries are aggregated into a unified evidence pool. This integration process involves three sequential operations.

- (i) **Deduplication:** When multiple sub-queries target overlapping information domains, the retrieved chunks may contain redundant content. The deduplication step identi-

- fies and merges semantically equivalent passages to prevent over-representation of specific evidence and reduce the token count passed to the generator.
- (ii) **Reranking:** The deduplicated chunks are reranked using a cross-encoder model, Cohere rerank-english-v3.0 (Cohere Inc., Toronto, ON, Canada), which evaluates each chunk's relevance against the original query rather than the decomposed sub-queries. This global reranking ensures that the final evidence set prioritizes information most relevant to the user's complete information need.
 - (iii) **Top-K (K = 10) selection:** The reranked chunks are truncated to fit within the LLM's context window, retaining only the highest-scoring passages. This selection step controls input token length, ensuring predictable response latency while maximizing information density in the generation prompt.

The final consolidated context, combined with the original query, is passed to the LLM generator to produce the answer. This structured integration pipeline—decompose, retrieve in parallel, prune, deduplicate, rerank, and generate—enables ToR-Lite to achieve improved retrieval coverage for multi-hop queries while maintaining the single LLM call characteristic of standard RAG systems. Figure 8 illustrates the context integration pipeline.

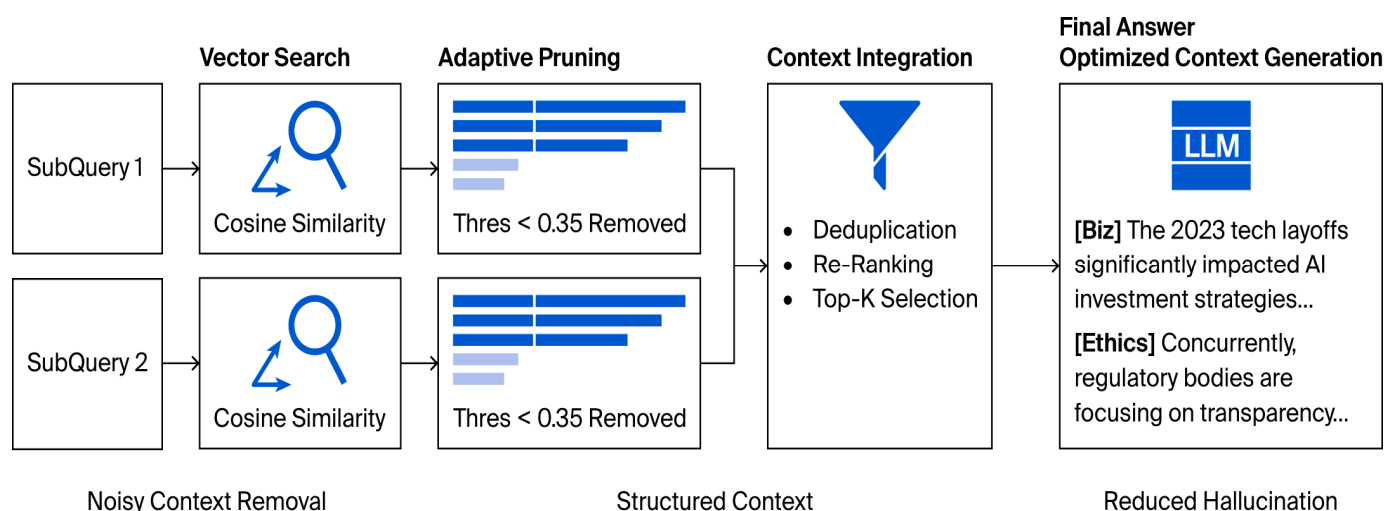


Figure 8. Context integration for final answer pipeline.

3.6. Experimental Setup

3.6.1. Hardware Environment

All experiments were conducted on a server equipped with dual NVIDIA H100 GPUs (80 GB VRAM each; NVIDIA Corporation, Santa Clara, CA, USA) to ensure consistent inference performance across all model configurations. The vector database was hosted on a separate node running Weaviate v1.25 (Weaviate B.V., Amsterdam, The Netherlands) with HNSW indexing for efficient approximate nearest neighbor search.

3.6.2. Dataset

We utilized the MultiHop-RAG benchmark [27], a dataset specifically designed to evaluate RAG systems on multi-hop reasoning tasks. The benchmark comprises 2556 queries derived from 609 news articles; of these, 301 null_query entries (control questions without valid gold answers) were excluded, yielding $n = 2255$ question–answer pairs for evaluation, with each question requiring evidence synthesis from at least two distinct sources. The questions are categorized into three types: inference (816 samples, 36.2%), comparison (856 samples, 37.9%), and temporal (583 samples, 25.9%). The corpus was processed using semantic chunking, yielding 4993 chunks with an average length of approximately

100 words per chunk. Extension to additional multi-hop benchmarks including HotpotQA [10], 2WikiMultiHopQA, and MuSiQue is identified as a priority for future work.

MultiHop-RAG was selected for three reasons. First, it directly targets the embedding dilution failure mode that ToR-Lite is designed to address, making it the most construct-valid benchmark for evaluating decomposition-based retrieval improvements. Second, its question types—inference, comparison, and temporal—span a range of multi-hop reasoning structures that stress-test different aspects of semantic boundary detection. Third, as a publicly available benchmark with standardized evaluation protocols, it enables reproducible comparisons with future work. The limitation of single-benchmark evaluation and the planned multi-benchmark validation are discussed in Section 5.6.

3.6.3. Models

The generator model was Qwen3-30B-A3B-Instruct (Alibaba Cloud, Hangzhou, China), configured with temperature 0.0 and maximum output tokens of 512 to ensure deterministic and reproducible outputs. For embedding generation and reranking, four configurations were compared to evaluate robustness across diverse embedding architectures, as illustrated in Table 2.

The four configurations span different dimensionalities, training objectives, and similarity score distributions. The BGE + Cohere configuration was identified as the primary configuration based on the results in Section 4.1; these selection decisions represent experimental design choices and are not claimed as methodological contributions.

3.6.4. Evaluation Metrics

Performance was evaluated using complementary metrics. As ToR-Lite is fundamentally a pre-retrieval optimization framework, Hits@10 is the most direct metric for evaluating its core contribution, as it isolates retrieval module performance from generator capability; Exact Match additionally captures downstream generation quality.

- **Exact Match (EM):** Measures generation accuracy by comparing the predicted answer against the ground truth after SQuAD-style normalization (lowercasing, article, and punctuation removal). A score of 1 is assigned only when the normalized prediction exactly matches the normalized answer.
- **Hits@10 (H@10):** Measures retrieval recall by checking whether the gold evidence fact appears as a substring within any of the top 10 retrieved chunks.
- **Latency:** Total end-to-end processing time in milliseconds, measured from query submission to answer generation. This metric captures the combined cost of decomposition, retrieval, pruning, reranking, and generation.
- **Decomposition Count (#Dq):** The number of sub-queries generated by the Word-Window Splitting algorithm ($1 \leq \#Dq \leq 3$). This metric enables analysis of the relationship between decomposition granularity and retrieval performance.

All results reported in Section 4—including the Baseline, ToR-Lite, and Adaptive ToR systems—were implemented and executed directly by the authors under identical experimental conditions. No results were imported from external literature. The three systems share the same retrieval infrastructure (Weaviate v1.25, HNSW indexing, BGE-base-en-v1.5 embeddings in the primary configuration), the same LLM generator (Qwen 3-30B-A3B-Instruct, temperature = 0.0, max tokens = 512), and the same evaluation procedure (SQuAD-style normalization for EM, substring matching for Hits@10). The only variable that differs across the three systems is the query processing stage: the Baseline performs single-step retrieval with no decomposition; ToR-Lite applies the Word-Window Splitting algorithm; and Adaptive ToR applies LLM-based complexity-aware tree decomposition.

This strict variable control ensures that all performance differences are attributable solely to the query decomposition mechanism.

4. Results

4.1. Overall Retrieval Performance

Table 3 presents the performance comparison across all evaluated configurations. ToR-Lite achieves consistent and statistically significant improvements in **retrieval performance (Hits@10)** over the Baseline across all four embedding–reranker configurations ($p < 0.001$ in all cases). For the primary configuration (BGE-base-en-v1.5 + Cohere rerank-english-v3.0), ToR-Lite improves Hits@10 by +6.03 percentage points and Exact Match (EM) by +0.89 pp. Although statistically significant, the EM effect size is negligible (Cohen’s $d < 0.06$), indicating that the primary contribution of ToR-Lite lies in retrieval coverage rather than answer-string precision.

Table 3. Performance comparison across embedding configurations.

Config	Metric	Baseline	ToR-Lite	Δ (TL)	Adaptive ToR	Δ (AT)	p -Value
BGE + Cohere	EM	66.52%	67.41%	+0.89 pp	68.38%	+1.86 pp	0.021
	H@10	38.26%	44.29%	+6.03 pp	51.84%	+13.58 pp	<0.001
BGE + msmarco	EM	65.37%	66.08%	+0.71 pp	67.01%	+1.64 pp	0.065
	H@10	34.06%	35.76%	+1.70 pp	38.67%	+4.61 pp	<0.001
Cohere + Cohere	EM	68.82%	68.34%	−0.48 pp	68.87%	+0.04 pp	0.324
	H@10	55.80%	57.38%	+1.58 pp	61.54%	+5.74 pp	<0.001
Mpnet + msmarco	EM	66.52%	65.99%	−0.53 pp	67.23%	+0.71 pp	0.201
	H@10	42.82%	44.84%	+2.02 pp	47.68%	+4.86 pp	<0.001

It is important to distinguish between the two evaluation metrics: Hits@10 improvements are consistent and significant across all four configurations ($p < 0.001$ in all cases), whereas EM improvements are configuration-dependent. EM reaches significance only for the primary BGE + Cohere setup (+0.89 pp, $p = 0.021$), while the remaining three configurations show either marginal gains or negligible decreases that do not reach statistical significance. This pattern reflects the nature of EM as a strict string-matching metric sensitive to answer phrasing, while Hits@10 directly measures whether relevant evidence chunks appear in the retrieved top-10 results—the more direct indicator of retrieval quality in multi-hop scenarios.

These results demonstrate cross-architecture robustness: the four configurations span three embedding families (BGE, Cohere, mpnet) with different dimensionalities (768d vs. 1024d), training objectives, and similarity score distributions. The consistency of Hits@10 improvements across all configurations ($p < 0.001$) confirms that the Word-Window Splitting algorithm generalizes across representationally diverse embedding spaces, and that the reported gains are attributable to the decomposition mechanism rather than to favorable interactions with a specific embedding model.

4.2. Effect of Decomposition Count

To isolate the contribution of decomposition granularity from other system components, the results were stratified by the number of sub-queries generated per question (#Dq), as presented in Table 4.

Table 4. Performance stratified by decomposition count for the BGE + Cohere configuration.

#Dq	N (%)	BL EM	TL EM	Δ EM	BL H@10	TL H@10	Δ H@10
1	123 (5.5%)	61.79%	62.60%	+0.81 pp	52.71%	52.71%	-
2	973 (43.1%)	65.36%	65.47%	+0.10 pp	38.96%	44.61%	+5.65 pp
3	1159 (51.4%)	67.99%	69.54%	+1.55 pp	36.13%	43.13%	+7.00 pp
All	2255 (100%)	66.52%	67.41%	+0.89 pp	38.26%	44.29%	+6.03 pp

Note: The #Dq = 3 subgroup Δ EM (+1.55 pp) and Δ H@10 (+7.00 pp) are reported as post hoc stratified analyses. Because queries were not pre-assigned to subgroups by random allocation, these results should be interpreted as exploratory rather than confirmatory. The higher EM and H@10 in the #Dq = 3 subgroup (51.4% of all queries) may partly reflect query characteristics (e.g., longer, more syntactically complex queries) that are independently associated with both higher decomposition counts and higher answer accuracy.

H@10 improvement increases monotonically with decomposition count. Queries with #Dq = 1 show no statistically significant change in either metric (Δ H@10 $\approx$ 0.00 pp, $p = 0.91$), confirming that when #Dq = 1, ToR-Lite degenerates to standard RAG, making the H@10 delta a direct quantification of decomposition benefit isolated from other system variations.

The EM improvement is most pronounced at #Dq = 3 (+1.55 pp), confirming that broader evidence coverage translates to better answer generation for complex multi-hop questions. The observation that 94.5% of MultiHop-RAG queries generate two or more sub-queries (#Dq = 2:43.1%, #Dq = 3:51.4%) indicates that the system operates predominantly in its effective regime, with only 5.5% of queries ($n = 123$) remaining unsplit (#Dq = 1).

4.3. Performance by Question Type

We examined performance across the three question types defined in the MultiHop-RAG benchmark. Table 5 summarizes the results.

Table 5. Performance breakdown by question type (BGE + Cohere configuration).

Question Type	N	BL EM	TL EM	Δ EM	BL H@10	TL H@10	Δ H@10
Comparison	856	64.60%	66.00%	+1.40 pp	51.52%	59.91%	+8.39 pp
Inference	816	91.42%	92.52%	+1.10 pp	26.14%	30.99%	+4.85 pp
Temporal	583	34.48%	34.31%	−0.17 pp	35.73%	39.97%	+4.23 pp

- **Comparison questions** benefit most substantially from ToR-Lite’s decomposition approach, achieving +1.40 pp EM and +8.39 pp H@10 improvements. Comparison questions inherently contain two or more contrasting entities that naturally map to separate sub-queries, and the Word-Window Splitting algorithm reliably detects contrastive boundaries as similarity valleys, enabling independent retrieval for each entity.
- **Inference questions** show moderate gains (+1.10 pp EM, +4.85 pp H@10). The relatively modest H@10 improvement compared to comparison questions reflects that inference questions often require deep reasoning within a single topic rather than evidence aggregation across multiple domains, limiting the benefit of query expansion through decomposition. The high Baseline EM for inference questions (91.42%) combined with the lowest H@10 (26.14%) indicates that inference answers are predominantly short factoid responses amenable to exact string matching, which the LLM can generate from parametric knowledge even when gold evidence is not retrieved. This interpretation is consistent with the observation that inference questions account for 57.4% of all fully pruned cases (kept = 0) in the APM analysis, confirming that relevant evidence for this question type is inherently difficult to retrieve via dense vector search.

- **Temporal questions** present the only case of EM regression (−0.17 pp) despite positive H@10 improvement (+4.23 pp). This divergence indicates that while the sliding window approach successfully retrieves relevant temporal evidence, it occasionally splits queries at temporal sequence markers (e.g., “first. . . then. . .”), disrupting chronological dependencies that must be preserved for correct answer generation. The retrieved documents contain correct evidence, but temporal ordering constraints are not sufficiently propagated to the final generation prompt. This limitation is discussed further in Section 5.6.

4.4. Comparison with Rule-Based Baselines

To determine whether the observed improvements stem from semantic boundary detection specifically or merely from increased decomposition count, we introduce three rule-based splitting Baselines: (i) Sentence Boundary Split, which segments queries at sentence boundaries; (ii) Punctuation Split, which segments queries at punctuation marks; and (iii) Fixed-Window Split, which divides every query into equal-length token windows. Table 6 presents the results.

Table 6. Comparison of ToR-Lite against rule-based Baselines (BGE + Cohere configuration).

Method	Avg. #Dq	EM	Δ EM	H@10	Δ H@10
Baseline	1.00	66.52%	-	38.26%	-
Sentence Boundary	1.04	66.21%	−0.31 pp	38.42%	+0.16 pp
Punctuation Split	2.19	66.83%	+0.31 pp	44.16%	+5.90 pp
Fixed-Window Split	3.00	66.56%	+0.04 pp	41.96%	+3.70 pp
ToR-Lite	2.46	67.41%	+0.89 pp	44.29%	+6.03 pp

Despite achieving a higher average decomposition count (3.00 vs. 2.46), Fixed-Window Split underperforms compared to ToR-Lite by 2.33 pp in Hits@10 (+3.70 pp vs. +6.03 pp). Sentence Boundary Split yields the weakest results (Δ H@10 = +0.16 pp), confirming that indiscriminate splitting introduces noise rather than signal. Punctuation Split achieves near-equivalent H@10 (+5.90 pp) but trails in EM (+0.31 pp vs. +0.89 pp), suggesting that punctuation-based boundaries capture some semantic structure incidentally but fail to detect sub-clausal semantic transitions that lack explicit punctuation markers. These results collectively demonstrate that the performance gains of ToR-Lite are attributable to the precision of semantic boundary detection rather than to decomposition count alone.

4.5. Parameter Sensitivity Analysis

The Word-Window Splitting algorithm employs a single tunable parameter, s , which controls the sensitivity of the adaptive threshold ($\tau = \mu - s\sigma$). Values of s across the range [1.0, 3.0] in increments of 0.5 were evaluated. Table 7 reports the results for the BGE + Cohere configuration.

The parameter $s = 2.0$ yields the highest performance across both metrics (EM: +0.89 pp, H@10: +6.03 pp) and was selected as the default configuration prior to final evaluation. At $s \leq 1.5$, the threshold becomes overly sensitive, producing excessive splitting (avg. #Dq approaching 2.60–2.61) that introduces noise from low-quality semantic boundaries. At $s \geq 2.5$, the threshold becomes increasingly conservative, with the avg. #Dq dropping to 2.18 and 1.94, respectively, limiting the system’s capacity to exploit decomposition benefits. The robustness of $s = 2.0$ across a ± 0.5 neighborhood (Δ H@10 within 0.51 pp; $s = 1.5$: +5.89 pp, $s = 2.5$: +5.52 pp) indicates that the algorithm is not highly sensitive to small parameter perturbations.

Table 7. Sensitivity analysis of the threshold scaling parameter s (BGE + Cohere configuration).

s Value	Avg. #Dq	EM	Δ EM	H@10	Δ H@10
1	2.61	66.92%	+0.40 pp	44.16%	+5.90 pp
1.5	2.60	66.87%	+0.35 pp	44.15%	+5.89 pp
2.0	2.46	67.41%	+0.89 pp	44.29%	+6.03 pp
2.5	2.18	66.87%	+0.35 pp	43.78%	+5.52 pp
3.0	1.94	66.78%	+0.26 pp	43.13%	+4.87 pp
Baseline	1.00	66.52%	-	38.26%	-

4.6. Cross-Embedding Robustness

A cross-embedding agreement analysis was conducted to assess whether ToR-Lite’s decomposition decisions are consistent across different embedding models. Agreement—defined as identical #Dq assignment—was observed in 46.4% of cases across all configuration pairs. While this level of agreement indicates that decomposition decisions are embedding-sensitive, it does not undermine the validity of the within-configuration results reported in Sections 4.1–4.5, which use fixed, matched pairs of embedding and reranker models throughout.

The embedding sensitivity reflects the fact that semantic similarity scores—and therefore the positions of similarity valleys—vary depending on the embedding model’s representational geometry. Pairwise agreement rates range from 61.4% to 65.8% across embedding families, with BGE-Cohere showing the highest agreement and Cohere-mpnet the lowest. Despite these decomposition-level differences, H@10 improvements remain statistically significant ($p < 0.001$) across all four configurations (Table 3), indicating that the Word-Window Splitting algorithm reliably identifies beneficial decomposition points within each embedding’s representational space.

4.7. Computational Efficiency Analysis

Table 8 summarizes the latency breakdown for Baseline, ToR-Lite, and Adaptive ToR under the BGE + Cohere configuration, averaged across all 2255 test queries.

Table 8. Mean latency breakdown by system component (ms; BGE + Cohere configuration).

Component	Baseline (ms)	ToR-Lite (ms)	Adaptive ToR
Retrieval	135.7	244.6	131.1
Algorithm (Decomposition)	-	1849.2	7454.7
Final LLM	680.4	451.0	513.5
Total	816.2	2544.8	8099.4
LLM Calls	1	1	2–20

The decomposition step accounts for 72.7% of ToR-Lite’s total latency (1849.2 ms of 2544.8 ms), reflecting the cost of embedding computation across sliding windows. This is nonetheless 4.03 times faster than Adaptive ToR’s LLM-based decomposition (7454.7 ms). The LLM generation time is reduced from 680.4 ms (Baseline) to 451.0 ms under ToR-Lite, a consequence of the APM filtering out low-relevance chunks and thereby shortening the context provided to the generator. The efficiency gain validates the APM’s role in not only improving retrieval quality but also optimizing downstream generation costs.

Adaptive ToR’s decomposition time (7454.7 ms, 92.0% of total) is dominated by iterative LLM calls for depth assignment, child expansion, and LLM-as-a-Judge evaluation.

ToR-Lite operates 3.18 times faster than Adaptive ToR; this latency advantage, combined with equivalent LLM call count to the Baseline, establishes ToR-Lite as the preferred choice for latency-sensitive deployments where LLM API costs constitute a primary operational constraint.

4.8. Failure Case Analysis

Among 2255 test queries, ToR-Lite produced degraded results—defined as $\Delta\text{EM} < -1.0$ pp and $\Delta\text{H@10} < 0.0$ pp simultaneously—in **19 cases (0.84%)**, representing genuine failure instances rather than metric-specific fluctuations. These cases were manually annotated to identify root causes; Table 9 summarizes the distribution.

Table 9. Failure case taxonomy.

(a) Split Quality Axis (<i>n</i> = 19).			
Failure Category	Count	% of 19	Description
Bad_split_quality	10	52.6%	Mid-sentence split; prepositional phrase; Temporal marker
Evidence_displacement	9	47.4%	Correct split; Reranker displaces gold evidence
(b) Mechanism Axis (<i>n</i> = 19).			
Failure Category	Count	% of 19	Description
Evidence_displacement	11	57.9%	Split is OK; New evidence displaces gold in reranking
Temporal_ dependency_broken	8	42.1%	Temporal relationship broken by split

Bad split quality is the dominant failure mode in the split quality axis (Table 9a), occurring when the similarity valley detection algorithm identifies a boundary within a semantically atomic unit—such as a named entity spanning multiple tokens, a prepositional phrase, or a temporal marker. In these cases (10 of 19; 52.6%), the resulting sub-queries are linguistically ill-formed or semantically incomplete, leading to retrieval of off-target passages. The remaining nine cases (47.4%) involve structurally correct splits where the reranker subsequently displaces gold evidence with topically adjacent but evidentially incorrect chunks.

Examined from the mechanism axis (Table 9b), evidence displacement is the most frequent category (11 of 19; 57.9%), occurring when the split itself is valid but newly retrieved candidates outrank the gold evidence in the reranking stage. Temporal dependency breaks account for eight cases (42.1%), arising when chronologically ordered query clauses are segmented at temporal sequence markers, disrupting the ordering signal necessary for correct answer generation.

These failure patterns point to three concrete directions for future improvement: (i) named-entity and idiom-aware boundary detection to prevent mid-sentence and mid-phrase splits; (ii) temporal dependency tagging to suppress decomposition of chronologically ordered query clauses; and (iii) reranking-stage gold evidence protection to reduce evidence displacement in cases where the split quality is not the root cause.

5. Discussion

5.1. Interpretation of Core Findings

The results presented in Section 4 demonstrate that LLM-free semantic decomposition is a viable and computationally efficient strategy for improving multi-hop retrieval. The primary failure mode of standard RAG on multi-hop queries is not reasoning failure at the generation stage, but retrieval failure caused by embedding dilution: when a complex query is encoded into a single vector, the resulting representation may fail to match strongly with evidence passages addressing specific sub-topics. By decomposing queries into semantically coherent units, ToR-Lite enables the vector space search to target specific, distinct pieces of evidence rather than relying on approximate matches against a diluted centroid.

The asymmetry between Hits@10 and Exact Match improvements warrants careful interpretation. Hits@10 improvements are consistent across all four embedding–reranker configurations ($p < 0.001$), indicating that ToR-Lite reliably surfaces relevant evidence chunks that the Baseline fails to retrieve. EM improvements, by contrast, are statistically significant only for the primary BGE + Cohere configuration ($p = 0.021$, Cohen’s $d < 0.06$) and turn marginally negative for configurations where the Baseline already achieves high answer-string accuracy.

The #Dq = 3 subgroup improvement ($\Delta\text{EM} = +1.55$ pp, $\Delta\text{H@10} = +7.00$ pp) reported in Section 4.2 is a post hoc stratified result and should be interpreted as exploratory rather than confirmatory, as larger gains in this subgroup may partly reflect query characteristics—such as greater syntactic complexity—that independently associate with both higher decomposition counts and higher answer accuracy.

5.2. Semantic Boundary Detection vs. Decomposition Count

The rule-based Baseline comparison (Section 4.4) provides the clearest evidence that ToR-Lite’s gains derive from boundary detection precision rather than decomposition count. Fixed-Window Split achieves a higher average #Dq (3.00) than ToR-Lite (2.46) yet delivers substantially lower Hits@10 improvement (+3.70 pp vs. +6.03 pp). Sentence Boundary Split, with an average #Dq of only 1.04, yields negligible improvement ($\Delta\text{H@10} = +0.16$ pp), confirming that splitting without decomposition into multiple sub-queries provides no retrieval benefit. These findings confirm the central hypothesis motivating the Word-Window Splitting design: the location of split points matters more than their quantity, and indiscriminate splitting introduces retrieval noise by generating sub-queries that lack coherent semantic targets.

Punctuation Split presents a more nuanced comparison, achieving near-equivalent H@10 (+5.90 pp vs. +6.03 pp) but trailing substantially in EM (+0.31 pp vs. +0.89 pp). Punctuation-based boundaries capture a substantial proportion of genuine semantic transitions in the MultiHop-RAG benchmark—a dataset whose queries originate from news articles and therefore tend to follow syntactically regular structures. However, Punctuation Split is structurally incapable of detecting semantic boundaries within clauses that lack explicit punctuation delimiters, a limitation that becomes increasingly consequential as query complexity and linguistic diversity increase.

Direct quantitative comparison with LLM-based external methods (IRCoT, Self-Ask, POQD) is not feasible on MultiHop-RAG, as none have been evaluated on this benchmark in the original literature and controlled reproduction would introduce confounding variables—generator model, retrieval configuration, and embedding architecture—that would compromise experimental validity. Table 1 therefore provides a methodological characteristic comparison rather than a quantitative one. All of the quantitative comparisons in Section 4 are conducted under strictly identical conditions, ensuring that performance dif-

ferences are attributable solely to the query decomposition mechanism. Multi-benchmark reproduction of selected external methods is planned as future work (Section 5.6).

5.3. Efficiency and the Pareto Frontier

Table 10 presents the Pareto frontier analysis of cost-performance positioning across all three systems.

Table 10. Pareto frontier analysis: cost-performance positioning of all three systems.

Method	EM	H@10	Latency	LLM Calls	$\Delta H@10$ Gain/ms
Baseline	66.52%	38.26%	~816 ms	1	-
ToR-Lite	67.41%	44.29%	~2545 ms	1	+0.0035 pp/ms
Adaptive ToR	68.38%	51.84%	~8099 ms	2~20	+0.0019 pp/ms

Note: The $\Delta H@10$ gain/ms metric in Table 10 is computed as $\Delta H@10 / (\text{Method Latency} - \text{Baseline Latency})$, representing the retrieval improvement obtained per millisecond of additional computational cost incurred beyond the Baseline.

ToR-Lite delivers approximately 1.9 times the retrieval improvement per unit of computational cost compared to Adaptive ToR (+0.0035 pp/ms vs. +0.0019 pp/ms), operating at one-third the latency while capturing 44% of its Hits@10 gain. In cloud-based AI services where inference costs scale with both computational time and LLM API call frequency, ToR-Lite reduces the per-query cost along two dimensions simultaneously: it eliminates LLM calls at the decomposition stage and reduces LLM generation time (680.4 ms $\rightarrow$ 451.0 ms) by shortening the context fed to the generator through APM pruning.

Adaptive ToR and ToR-Lite are not directly competing alternatives but rather occupy different positions on the accuracy-latency trade-off curve. Adaptive ToR remains the superior choice when latency budgets permit and maximum retrieval accuracy is the primary objective, for example, in offline document analysis or high-value, low-frequency query processing. ToR-Lite is the appropriate choice when latency constraints are binding, LLM API costs are prohibitive, or deployment infrastructure does not support multi-call LLM decomposition, as is typical of large-scale SaaS applications processing thousands of queries per day.

5.4. Positioning Within Research Lineage

ToR-Lite completes a three-stage progression in the authors' Tree-of-Retrieval research progression. ToR-RAG [17] established the effectiveness of hierarchical query decomposition but employed fixed-depth expansion, imposing a constant maximum computational cost regardless of query complexity. Adaptive ToR [18] introduced dynamic depth control through LLM-based complexity assessment, achieving superior accuracy at sustained LLM dependency. ToR-Lite resolves this residual dependency by demonstrating that effective decomposition can be achieved through purely geometric analysis of embedding representations, occupying a Pareto-superior position for the dominant real-time deployment scenario.

5.5. Theoretical Grounding

The Word-Window Splitting algorithm is grounded in three established theoretical frameworks. First, it draws from TextTiling [23], which demonstrated that coherent textual units produce higher within-segment lexical cohesion than cross-segment transitions—a property that ToR-Lite operationalizes in the embedding space rather than the lexical space. Second, the adaptive threshold mechanism ($\tau = \mu - s\sigma$) formalizes the intuition from distributional semantics that semantic similarity scores follow query-specific distributions,

providing principled statistical grounding for the choice of $s = 2.0$ (see Section 3.2). Third, the overall LLM-free decomposition architecture aligns with the LlamaIndex modular RAG paradigm [9], which advocates for decoupling retrieval-stage components from generation-stage LLM dependencies to improve system predictability and reduce inference costs.

5.6. Limitations and Future Directions

5.6.1. Limitations

Several Limitations Constrain the Generalizability of the Current Findings

First, all experiments were conducted on the MultiHop-RAG benchmark, a single dataset derived from English-language news articles whose syntactically regular query distribution may favor ToR-Lite's boundary detection patterns relative to more diverse corpora.

Second, the cross-embedding agreement analysis (Section 4.6) reveals that decomposition decisions are embedding-sensitive: only 46.4% of queries receive identical #Dq assignments across all three embedding models, necessitating recalibration of s when the embedding backbone is updated in production.

Third, the EM metric is a conservative lower bound on true answer quality in multi-hop scenarios; future evaluations should supplement EM with F1-score and, where feasible, human-judged answer quality assessments.

Fourth, the 5.4% of queries triggering the APM fallback represent cases where ToR-Lite provides no added value over the Baseline; improving decomposition coverage for this subgroup would require complementary strategies such as NER-based entity separation or WH-word pattern matching.

Fifth, ToR-Lite's parallel retrieval architecture is designed for additive multi-hop queries whose sub-topics are semantically independent, and does not handle conditional hop relationships where one sub-query's answer is a prerequisite for the next retrieval step. Sequential dependency detection as a pre-decomposition classifier is planned for future work (Section 5.6).

Sixth, the window size parameters ($W = 7$, $W = 9$) were calibrated on English data and may not generalize to agglutinative languages (e.g., Korean, Japanese, Turkish) or isolating languages (e.g., Chinese), where morphological or character-level boundaries constitute the natural segmentation unit. Multilingual validation is required before deploying ToR-Lite in non-English environments.

5.6.2. Future Directions

Building on the limitations identified above, six directions for future research are proposed. First, multi-benchmark validation on HotpotQA [10], 2WikiMultiHopQA, and MuSiQue would strengthen generalizability claims beyond the single-benchmark scope of the current study. Second, a lightweight temporal and conditional dependency detector could suppress decomposition of chronologically ordered queries, directly addressing the dominant failure mode identified in Section 4.8. Third, multilingual evaluation is required to assess whether the English-calibrated window parameters ($W = 7$, $W = 9$) generalize to agglutinative and isolating languages. Fourth, named-entity-aware boundary correction, augmenting valley detection with an NER pass [35] grounded in frame-semantic analysis [33], would prevent mid-entity splitting. Fifth, sub-query result ordering—a post-retrieval step sequencing evidence according to the original query's logical structure—could improve answer quality for sequentially dependent queries. Sixth, multi-generator validation using diverse LLM architectures (e.g., GPT-4o, Llama-3, Gemma) would strengthen the generalizability of EM results, which currently rely on a single generator. While Hits@10 evaluates retrieval quality independently of the generator, EM may vary across models with different parametric knowledge and answer phrasing tendencies.

6. Conclusions

Experiments on the MultiHop-RAG benchmark ($n = 2255$) demonstrated statistically significant improvements over a strong RAG Baseline: +6.03 pp Hits@10 ($p < 0.001$) and +0.89 pp Exact Match ($p = 0.021$, 95% CI [0.18, 1.60]) under the primary BGE + Cohere configuration, with Hits@10 gains robust across all four embedding–reranker configurations evaluated. The modest EM gain reflects ToR-Lite’s nature as a pre-retrieval framework whose primary contribution lies in retrieval coverage rather than answer-string precision. Rule-based Baseline comparisons confirm that performance improvements stem from semantic boundary detection precision rather than decomposition count alone. ToR-Lite delivers approximately 1.9 times the retrieval improvement per unit of computational cost compared to LLM-based Adaptive ToR (+0.0035 pp/ms vs. +0.0019 pp/ms), operating at one-third the latency while capturing 44% of its retrieval gain.

The primary limitations of this study are the single-benchmark evaluation scope and the embedding sensitivity of decomposition decisions. Future work will focus on multi-benchmark validation (HotpotQA, 2WikiMultiHopQA, MuSiQue), temporal and conditional dependency detection, named-entity-aware boundary correction, and multilingual extension. ToR-Lite’s deterministic, single-LLM-call architecture positions it as a practical solution for latency-sensitive, cost-constrained cloud AI deployments where LLM-based decomposition is computationally prohibitive.

Author Contributions: Conceptualization H.-K.Y. and N.M.; methodology, H.-K.Y.; software, H.-K.Y.; validation, W.K. and N.M.; formal analysis, H.-K.Y.; investigation, H.-K.Y.; resources, N.M.; data curation, H.-K.Y.; writing—original draft, H.-K.Y.; writing—review and editing, W.K. and N.M.; visualization, H.-K.Y.; supervision, N.M.; project administration, N.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The MultiHop-RAG dataset used in this study is publicly available at <https://github.com/yixuantt/MultiHop-RAG> (accessed on 10 February 2026). The code implementing the Word-Window Splitting algorithm and ToR-Lite framework is available from the corresponding author upon reasonable request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Singh, A.; Ehtesham, A.; Kumar, S.; Khoei, T. Agentic retrieval-augmented generation: A survey on agentic RAG. *arXiv* **2025**, arXiv:2501.09136. [CrossRef]
2. Yu, Z.; Zhang, Z.; Zeadally, S. Energy Efficient Hierarchical Federated Learning over Intelligent IIoT Devices. *Hum.-Centric Comput. Inf. Sci.* **2025**, *15*, 57. [CrossRef]
3. Byun, J.; Seo, S.; Bae, K.J.; Kim, E.S. Deep Learning-Based Decision Support System for Identifying Emerging Items in National Strategic Technology Fields. *Hum.-Centric Comput. Inf. Sci.* **2025**, *15*, 48. [CrossRef]
4. Chen, K.; Nadig, R.; Frouzakis, M.; Mansouri Ghiasi, N.; Liang, Y.; Mao, H.; Park, J.; Sadrosadati, M.; Mutlu, O. REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing. In Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA’25), Tokyo, Japan, 21–25 June 2025; pp. 1171–1192. [CrossRef]
5. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.; Rocktäschel, T.; et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 9459–9474.
6. Izacard, G.; Lewis, P.; Lomeli, M.; Hosseini, L.; Petroni, F.; Schick, T.; Dwivedi-Yu, J.; Joulin, A.; Riedel, S.; Grave, E. Atlas: Few-Shot Learning with Retrieval Augmented Language Models. *J. Mach. Learn. Res.* **2023**, *24*, 11912–11954.
7. Yan, S.; Gu, J.; Zhu, Y.; Ling, Z. Corrective retrieval augmented generation. *arXiv* **2024**, arXiv:2401.15884. [CrossRef]

8. Jeong, S.; Baek, J.; Cho, S.; Hwang, S.J. Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. In *Proceedings of the NAACL-HLT 2024, Mexico City, Mexico, 16–21 June 2024*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2024; pp. 7036–7050.
9. Gao, Y.; Xiong, Y.; Gao, X.; Jia, K.; Pan, J.; Bi, Y.; Dai, Y.; Sun, J.; Wang, H. Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv* **2024**, arXiv:2312.10997. [\[CrossRef\]](#)
10. Yang, Z.; Qi, P.; Zhang, S.; Bengio, Y.; Cohen, W.W.; Salakhutdinov, R.; Manning, C.D. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP 2018), Brussels, Belgium, 31 October–4 November 2018*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2018; pp. 2369–2380.
11. Karpukhin, V.; Oğuz, B.; Min, S.; Lewis, P.; Wu, L.; Edunov, S.; Chen, D.; Yih, W. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020), Online, 16–20 November 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 6769–6781.
12. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the EMNLP 2019, Hong Kong, China, 3–7 November 2019*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2019; pp. 3980–3990.
13. Zhou, D.; Schärli, N.; Hou, L.; Wei, J.; Scales, N.; Wang, X.; Schuurmans, D.; Cui, C.; Bousquet, O.; Le, Q.; et al. Least-to-Most Prompting Enables Complex Reasoning in Large Language Models. In *Proceedings of the ICLR 2023, Kigali, Rwanda, 1–5 May 2023*; pp. 1–21.
14. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.; Zhou, D. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 24824–24837.
15. Trivedi, H.; Balasubramanian, N.; Khot, T.; Sabharwal, A. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In *Proceedings of the ACL 2023, Toronto, ON, Canada, 9–14 July 2023*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2023; pp. 10014–10037.
16. Press, O.; Zhang, M.; Min, S.; Schmidt, L.; Smith, N.A.; Lewis, M. Measuring and Narrowing the Compositionality Gap in Language Models. In *Proceedings of the Findings of EMNLP 2023, Singapore, 6–10 December 2023*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2023; pp. 5687–5711.
17. Yoo, H.-K.; Moon, N. ToR-RAG: A Tree-of-Retrieval-based Retrieval-Augmented Generation for Complex Question Processing. *J. Korea Soc. Comput. Inf.* **2025**, *30*, 23–31.
18. Yoo, H.-K. Design and Verification of Adaptive ToR Structure via Query Complexity Awareness. Ph.D. Thesis, Hoseo University, Seoul, Republic of Korea, February 2026. Available online: <http://www.dcollection.net/handler/hoseo/200000961944> (accessed on 19 March 2026).
19. Liu, Y.; Li, J.; Wu, Y.; Chen, Z. POQD: Performance-oriented query decomposer for multi-vector retrieval. In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Vancouver, BC, Canada, 13–19 July 2025*; PMLR: New York, NY, USA, 2025; Volume. 267.
20. Wang, L.; Yang, N.; Wei, F. Query2Doc: Query Expansion with Large Language Models. In *Proceedings of the EMNLP 2023, Singapore, 6–10 December 2023*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2023; pp. 9414–9423.
21. Gupta, S.; Ranjan, R. A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions. *arXiv* **2024**, arXiv:2410.12837.
22. Xu, Z.; Mo, F.; Huang, Z.; Zhang, C.; Yu, P.; Wang, B.; Srikumar, V. A Survey of Model Architectures in Information Retrieval. *arXiv* **2025**, arXiv:2502.14822. [\[CrossRef\]](#)
23. Hearst, M.A. TextTiling: Segmenting Text into Multi-paragraph Subtopic Passages. *Comput. Linguist.* **1997**, *23*, 33–64.
24. Kiss, C.; Nagy, M.; Szilágyi, P. Max–Min Semantic Chunking of Documents for RAG Application. *Discov. Comput.* **2025**, *28*, 138. [\[CrossRef\]](#)
25. Malkov, Y.; Yashunin, D. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* **2020**, *42*, 824–836. [\[CrossRef\]](#) [\[PubMed\]](#)
26. Kim, K.; Kim, J.; Chung, H.; Cha, M.-H.; Kim, H.-Y.; Kim, Y. Cost-efficient LLM serving in the cloud: VM selection with KV cache offloading. *arXiv* **2025**, arXiv:2504.11816. [\[CrossRef\]](#)
27. Tang, Y.; Yang, Y. MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries. In *Proceedings of the COLM 2024, Philadelphia, PA, USA, 7–9 October 2024*; pp. 1–18.
28. Kostric, I.; Balog, K. A Surprisingly Simple yet Effective Multi-Query Rewriting Method for Conversational Passage Retrieval. In *Proceedings of the SIGIR 2024, Washington, DC, USA, 14–18 July 2024*; ACM: New York, NY, USA, 2024; pp. 1–10. [\[CrossRef\]](#)
29. Khattab, O.; Zaharia, M. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the SIGIR 2020, Xi'an, China, 25–30 July 2020*; ACM: New York, NY, USA, 2020; pp. 39–48.
30. Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Larson, J. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv* **2024**, arXiv:2404.16130. [\[CrossRef\]](#)

31. Lu, Y.; Wang, X.; Chen, H.; Li, J. KEDKG: Knowledge Editing with Dynamic Knowledge Graphs for Multi-hop Question Answering. In *Proceedings of the AAAI 2025, Philadelphia, PA, USA, 25 February–4 March 2025*; AAAI Press: Palo Alto, CA, USA, 2025; pp. 1–9.
32. Casanueva, I.; Temčinas, T.; Gerber, D.; Agez, M.; Henderson, M. NLU++: A Multi-Label, Slot-Rich, Generalisable Dataset for Natural Language Understanding in Task-Oriented Dialogue. In *Proceedings of the Findings of NAACL 2022, Seattle, WA, USA, 10–15 July 2022*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2022; pp. 1998–2013.
33. Baker, C.F.; Fillmore, C.J.; Lowe, J.B. The Berkeley FrameNet project. In *Proceedings of the 36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics (COLING-ACL 1998), Montreal, QC, Canada, 10–14 August 1998*; Morgan Kaufmann Publishers: San Francisco, CA, USA, 1998; pp. 86–90. [[CrossRef](#)]
34. Nivre, J.; de Marneffe, M.-C.; Ginter, F.; Hajič, J.; Manning, C.D.; Pyysalo, S.; Schuster, S.; Tyers, F.; Zeman, D. Universal Dependencies v2: An evergrowing multilingual treebank collection. In *Proceedings of the Twelfth Language Resources and Evaluation Conference (LREC 2020), Marseille, France, 11–16 May 2020*; European Language Resources Association: Paris, France, 2020; pp. 4034–4043.
35. Honnibal, M.; Montani, I.; Van Landeghem, S.; Boyd, A. *spaCy: Industrial-Strength Natural Language Processing in Python*; Zenodo: Geneva, Switzerland, 2020. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.